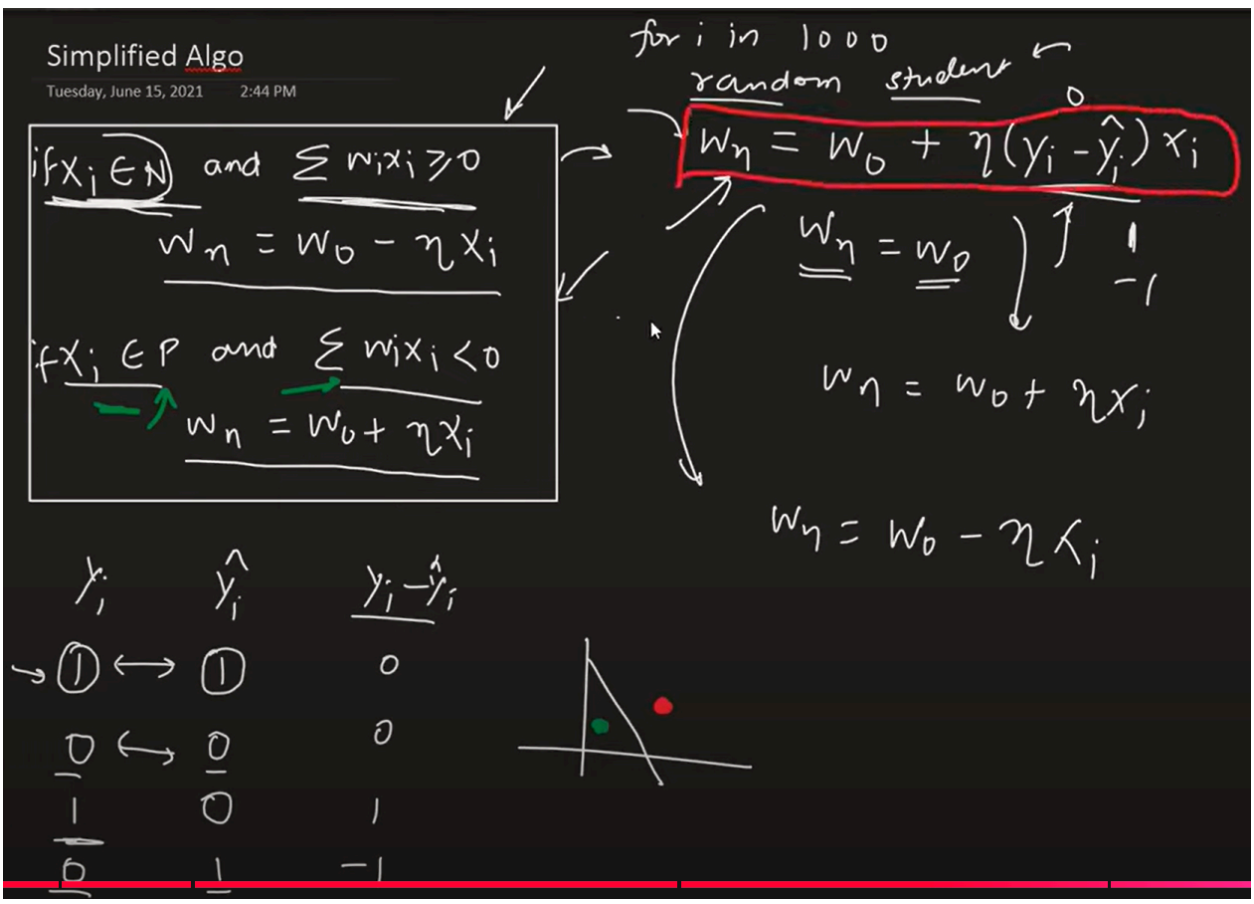
Logistic Regression

For
a negative a point in positive region → subtract

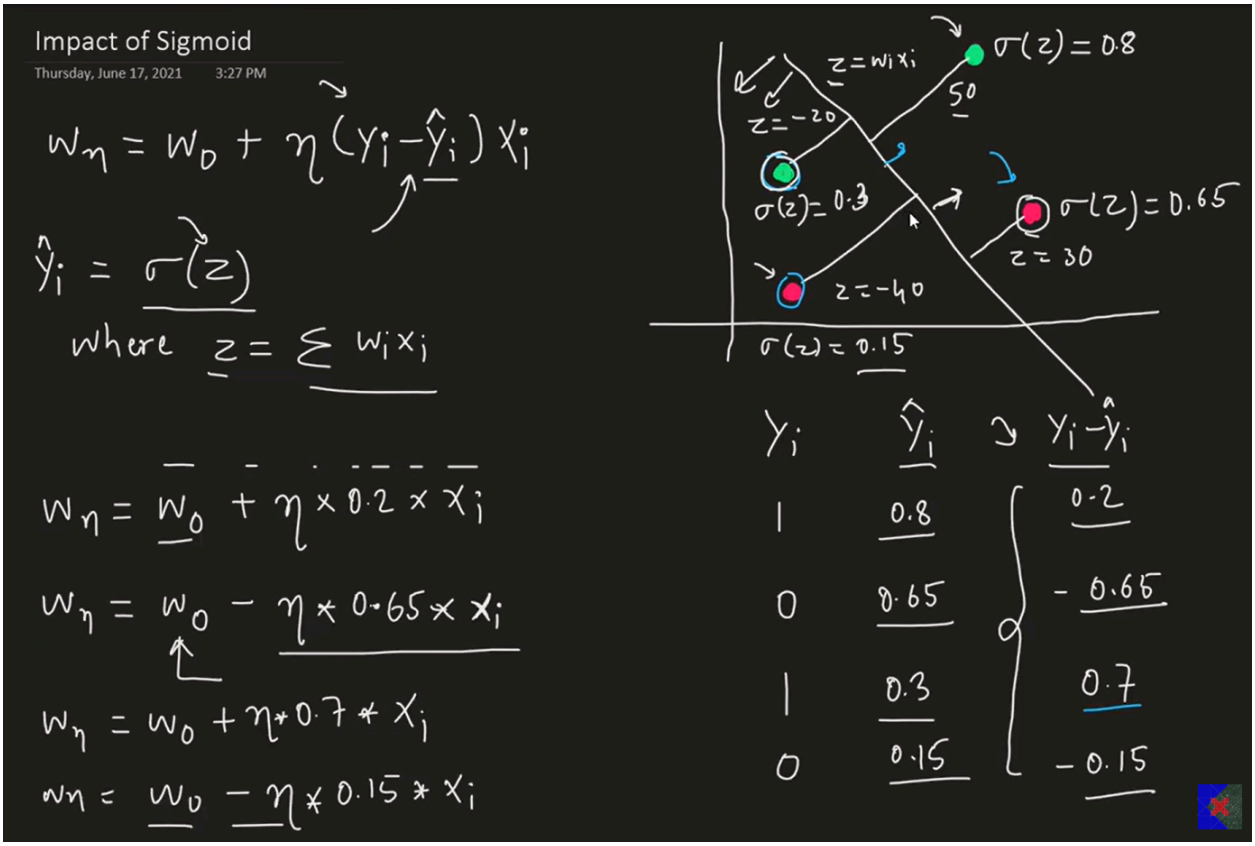
a positive point in negative region → add

simplified algo for a Logistic regression (also similar to gradient descent) except it changes on both cases with both adding and subtracting

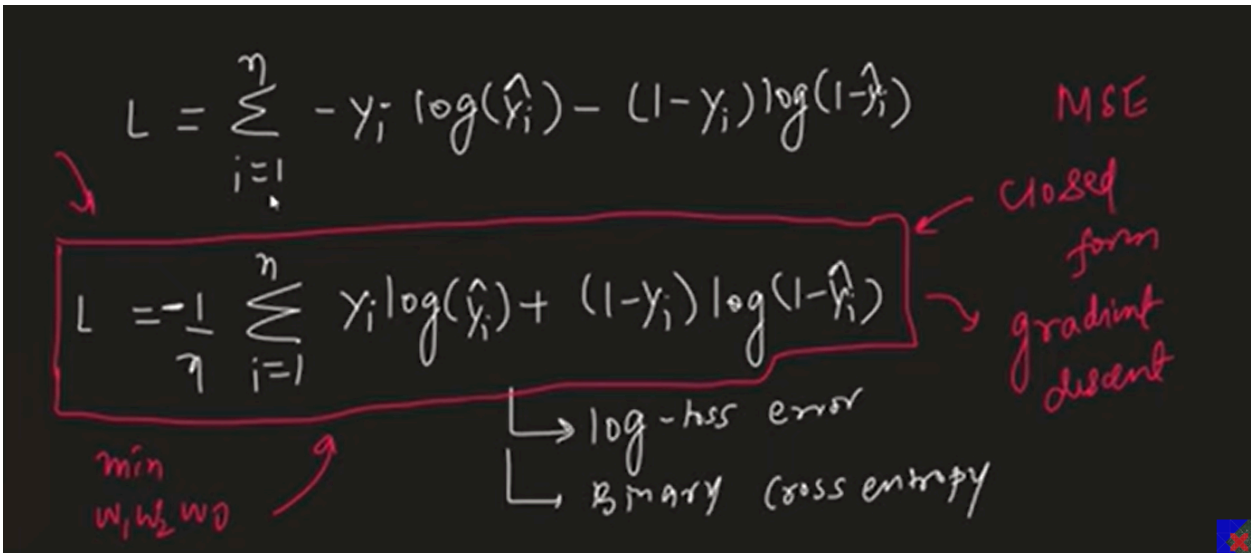


Perceptron trick Code ^

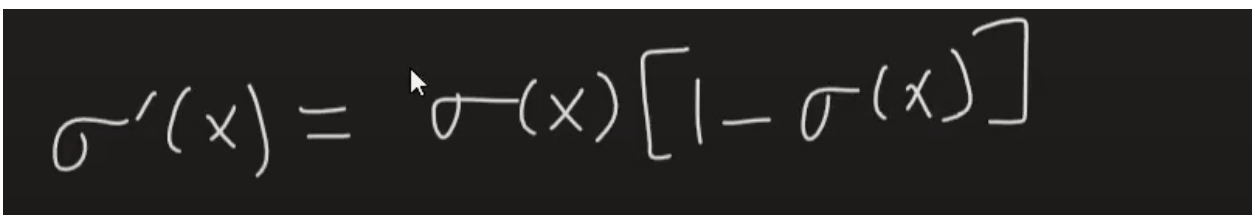
Using sigmoid



Maximum Likelihood



Derivative of a sigmoid function



Applying gradient decent and final code from scratch of Logistic regression

$$\sigma(w_1 x_{11} + w_2 x_{12} + w_3 x_{13} + \dots + w_n x_{1n} + w_0) = y_1$$

$$\sigma(w_1 x_{21} + w_2 x_{22} + w_3 x_{23} + \dots + w_n x_{2n} + w_0) = \hat{y}_2$$

$$\hat{y}$$

$$\hat{y} = \begin{bmatrix} \hat{y}_1 \\ \hat{y}_2 \\ \vdots \\ \hat{y}_m \end{bmatrix} = \begin{bmatrix} \sigma(w_0 + w_1 x_{11} + w_2 x_{12} + \dots + w_n x_{1n}) \\ \sigma(w_0 + w_1 x_{21} + w_2 x_{22} + \dots + w_n x_{2n}) \\ \vdots \\ \sigma(w_0 + w_1 x_{m1} + w_2 x_{m2} + \dots + w_n x_{mn}) \end{bmatrix}$$

$$\hat{y} = \sigma\left(\begin{bmatrix} w_0 + w_1 x_{11} + w_2 x_{12} + \dots + w_n x_{1n} \\ w_0 + w_1 x_{21} + w_2 x_{22} + \dots + w_n x_{2n} \\ \vdots \\ w_0 + w_1 x_{m1} + w_2 x_{m2} + \dots + w_n x_{mn} \end{bmatrix}\right)$$

$$\hat{y} = \sigma\left(\begin{bmatrix} 1 & x_{11} & x_{12} & \dots & x_{1n} \\ 1 & x_{21} & x_{22} & \dots & x_{2n} \\ \vdots & & & & \\ 1 & x_{m1} & x_{m2} & \dots & x_{mn} \end{bmatrix} \begin{bmatrix} w_0 \\ w_1 \\ \vdots \\ w_n \end{bmatrix}\right)$$

Loss function comes L

$$\boxed{\hat{y} = \sigma(XW)}$$

$$L = -\frac{1}{m}\sum_{i=1}^{m} y_i \log(\hat{y}_i) + (1-y_i)\log(1-\hat{y}_i)$$

$$L = -\frac{1}{m}\left[\sum_{i=1}^{m} y_i \log(\hat{y}_i) + \sum_{i=1}^{m} (1-y_i)\log(1-\hat{y}_i)\right]$$

$$\sum_{i=1}^{m} y_i \log(\hat{y}_i) = y_1 \log \hat{y}_1 + y_2 \log \hat{y}_2 + y_3 \log \hat{y}_3 + \dots + y_m \log \hat{y}_m$$

$$\begin{bmatrix} y_1 & y_2 & y_3 & \cdots & y_m \end{bmatrix} \begin{bmatrix} \log \hat{y}_1 \\ \log \hat{y}_2 \\ \vdots \\ \log \hat{y}_m \end{bmatrix}$$

$$\boxed{Y \log \hat{Y} = Y \log(\sigma(XW))}$$

$$L = -\frac{1}{m}\left[y \log \hat{y} + (1-y)\log(1-\hat{y}) \right]$$

where $\hat{y} = \sigma(XW)$

GD [W] find

Loss function in Matrix form

$$L = -\frac{1}{m}\left[y \log(\sigma(wx)) + (1-y)\log(1-\sigma(wx)) \right]$$

minimum

gradient descent

$W = [\quad]$

$w_0 \rightarrow w_n$

for i in epochs:

$$w = w - \eta \frac{\Delta L}{\Delta w}$$

$(n+1)$

$$\frac{\Delta L}{\Delta w} = \left[\frac{\partial L}{\partial w_0}, \frac{\partial L}{\partial w_1}, \frac{\partial L}{\partial w_2}, \cdots \frac{\partial L}{\partial w_n} \right]$$

$$\frac{d}{dw} y \log \hat{y} \Rightarrow y \frac{d}{dw} \log \hat{y} \Rightarrow \frac{y}{\hat{y}} \frac{d}{dL}(\hat{y})$$

$$\frac{y}{\hat{y}} \frac{d}{dL} \sigma(wx) \Rightarrow \frac{y}{\hat{y}} \sigma(wx)[1-\sigma(wx)] \frac{d}{dw} wx$$

$$= \frac{y}{\cancel{\hat{y}}} \cancel{\hat{y}}(1-\hat{y})\, x = y(1-\hat{y})\, x$$

$$\frac{d}{dw}(1-y)\log(1-y) \Rightarrow (1-y)\frac{d}{dw}\log(1-\hat{y}) \Rightarrow \frac{(1-y)}{(1-\hat{y})}\frac{d}{dw}[1-\hat{y}]$$

$$= -\frac{(1-y)}{(1-\hat{y})}\frac{d}{dw}\sigma(wx) \Rightarrow -\frac{(1-y)}{(1-\hat{y})}\left[\sigma(wx)[1-\sigma(wx)]\right]$$

$$\Rightarrow -\frac{(1-y)}{\cancel{(1-\hat{y})}}\hat{y}\cancel{(1-\hat{y})}\, x = \boxed{-\hat{y}(1-y)\, x}$$

$$\frac{dL}{dw} = -\frac{1}{m}\left[y(1-\hat{y})x - \hat{y}(1-y)x\right]$$

$$= -\frac{1}{m}\left[y(1-\hat{y}) - \hat{y}(1-y)\right]x$$

final loss function derivative with respective to w for gradient descent

$$= -\frac{1}{m}\left[y - \cancel{y\hat{y}} - y + \cancel{y\hat{y}} \right] x$$

$$\boxed{\frac{\Delta L}{\Delta w} = -\frac{1}{m}(y-\hat{y})\,x}$$

Final formula for logistic regression using gradient descent , sigmoid function

$$\boxed{\underline{w} = \underline{w} + \eta \frac{1}{m}(y-\hat{y})\,x}$$

```
from sklearn.datasets import make_classification
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LogisticRegression

X, y = make_classification(n_samples=100, n_features=2, n_inf
ormative=1, n_redundant=0,
                           n_classes=2, n_clusters_per_class=
1, random_state=41,
                           hypercube=False, class_sep=20)
```

```
plt.figure(figsize=(10, 6))
plt.scatter(X[:, 0], X[:, 1], c=y, cmap='winter', s=100)

# Fix: use Python None (not string) for 'penalty'
lor = LogisticRegression(penalty=None, solver='lbfgs', max_it
er=1000)
lor.fit(X, y)


print(lor.coef_)
print(lor.intercept_)

m1 = -(lor.coef_[0][0] / lor.coef_[0][1])
b1 = -(lor.intercept_[0] / lor.coef_[0][1])

x_input = np.linspace(-3, 3, 100)
y_input = m1 * x_input + b1

# Move sigmoid function above its usage
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def gd(X, y):
    X = np.insert(X, 0, 1, axis=1)
    weights = np.ones(X.shape[1])
    lr = 0.7

    for i in range(12000):
        y_hat = sigmoid(np.dot(X, weights))
        weights = weights + lr * (np.dot((y - y_hat), X) / X.
shape[0])

    return weights[1:], weights[0]

coef_, intercept_ = gd(X, y)
```

```
m = -(coef_[0] / coef_[1])
b = -(intercept_ / coef_[1])
x_input1 = np.linspace(-3, 3, 100)
y_input1 = m * x_input1 + b

plt.figure(figsize=(10, 6))
plt.plot(x_input, y_input, color='red', linewidth=3)
plt.plot(x_input1, y_input1, color='black', linewidth=3)
plt.scatter(X[:, 0], X[:, 1], c=y, cmap='winter', s=100)
plt.ylim(-3, 2)

np.insert(X, 0, 1, axis=1)

X1 = np.insert(X, 0, 1, axis=1)
np.ones(X1.shape[1])
```